

HW1: Code Profiling, Optimization, and HWSW Understanding

Objective

The goal of this homework is to help you practice applying software optimizations by using your understanding of how the underlying hardware works. Through practical use of performance profiling tools and code-level optimizations. You will be required to demonstrate how changes in code (based on system understanding) can lead to measurable performance improvements, while preserving the correctness of the program.

Assignment Instructions

You must select or design a program (written in C or C++) that performs a meaningful task of your choice. Your creativity is welcome—we encourage you to be imaginative, as long as your program allows for interesting profiling and optimization opportunities.

You are also welcome to use AI tools (like ChatGPT or Copilot) to help generate your program. If you choose this route, please include the prompts you used and make sure you understand the resulting code. For example, to create the second example below, we used the prompt:

“Write C++ code for an n-body simulation with gravity forces between particles and picosecond time resolution.”

To help guide you, here are two example project ideas at the level of complexity we’re aiming for:

1. **Key-Value Store:** Implement a basic hash table (without using STL). First insert 100 million key-value pairs, then perform a series of read/write operations.
2. **Particle System Simulation:** Maintain a list of particles, each with a mass and a 2D position. In each iteration:
 - a. **A.** For each particle, compute the total force and direction from all other particles (naïve $O(N^2)$ approach).
 - b. **B.** Use that force to update the particle’s velocity and position over a 1 picosecond step.
 - c. Repeat A and B for a given number of iterations.

You can choose one of these ideas, adapt them, or design a different task entirely—just make sure it’s of a similar scale and complexity.

1. Run the program twice:

- Once in an **unoptimized** version (naive/easy implementation).
- Once in an **optimized** version, where you apply a specific code or system-level optimization. We encourage you to use techniques discussed in class, such as improving IPC, enhancing cache behavior, algorithmic adaptations/improvements, and more.

2. Ensure that **input and output remain identical** between the unoptimized and optimized versions.
3. Use **perf** to analyze performance in both versions.
4. Evaluate and explain the observed performance difference.

Submission Requirements

Submit a .zip file containing **all the following components**:

1. **PDF document** (maximum 3 pages) explaining:
 - Your general approach and thought process.
 - Description of your **unoptimized implementation**:
 - Why did you pick it.
 - What does the program do?
 - Why is it not optimized?
 - Results from profiling (e.g., runtime, cache misses, CPU cycles, etc.).
 - Description of your **optimization**:
 - What change did you make?
 - Why did you choose this specific optimization?
 - What system/hardware-level insight led you to it?
 - What were the profiling results?
 - Were the results expected or surprising?
 - A **comparison** between the two profiling results:
 - Highlight key improvements or regressions.
 - Provide **reasoning** for observed behavior.
 - Convince the reader (the course staff) why the performance gain makes sense (your explanation should be in level, that a graduated ECE student will understand).

You are also encouraged to describe **approaches that didn't work** or optimizations you considered but dropped. This reflects your exploration and critical thinking.

2. **C or C++ source file** of the **unoptimized version**.
3. **C or C++ source file** of the **optimized version**.

4. **Shell script (.sh)** containing the exact commands you used for:
 - Compiling the code.
 - Running the code.
 - Profiling with perf or any other profiling tool.
5. **PDF** file including the names and IDs of the submitters.

Grading Criteria

Submission completeness and correctness. All required files are submitted. The explanation is clear and coherent. Profiling tool is used correctly. Optimization works and is justified.	70%
Complexity, difficulty, and originality How non-trivial was your chosen problem? How clever or insightful was your optimization? How unique is your solution compared to others students?	30%

You **must not**:

1. Use examples shown in **class lectures or tutorials**.

Violation of this rule may result in a grade of zero.

We encourage you to:

1. Demonstrate creativity and original thinking.
2. Focus on understanding the hardware/software interplay.
3. Use profiling data to support your arguments.
4. Use AI tools for both code generation and optimization. **However**, we require that you document the prompts you used and make sure you can clearly explain the optimizations you applied.

The course staff will be available to assist you on the course forum.

